
flask-jwt-simple Documentation

Release 0.0.3

vimalloc rlam3

Nov 17, 2018

Contents

1	Installation	3
2	Basic Usage	5
3	Changing JWT Claims	7
4	Changing Default Behaviors	9
5	Configuration Options	11
6	API Documentation	13
6.1	Configuring JWT Options	13
6.2	Protected endpoint decorators	14
6.3	Utilities	14
	Python Module Index	17

In here you will find examples of how to use Flask JWT Simple.

CHAPTER 1

Installation

The easiest way to start working with this extension with pip:

```
$ pip install flask-jwt-simple
```

If you want to use asymmetric (public/private key) key signing algorithms, include the `asymmetric_crypto` extra requirements.

```
$ pip install flask-jwt-simple[asymmetric_crypto]
```

Note that if you are using ZSH (possibly other shells too), you will need to escape the brackets

```
$ pip install flask-jwt-simple\[asymmetric_crypto\]
```

If you prefer to install from source, you can clone this repo and run

```
$ python setup.py install
```


CHAPTER 2

Basic Usage

In its simplest form, there is not much to using flask_jwt_simple.

```
from flask import Flask, jsonify, request
from flask_jwt_simple import (
    JWTManager, jwt_required, create_jwt, get_jwt_identity
)

app = Flask(__name__)

# Setup the Flask-JWT-Simple extension
app.config['JWT_SECRET_KEY'] = 'super-secret' # Change this!
jwt = JWTManager(app)

# Provide a method to create access tokens. The create_jwt()
# function is used to actually generate the token
@app.route('/login', methods=['POST'])
def login():
    if not request.is_json:
        return jsonify({"msg": "Missing JSON in request"}), 400

    params = request.get_json()
    username = params.get('username', None)
    password = params.get('password', None)

    if not username:
        return jsonify({"msg": "Missing username parameter"}), 400
    if not password:
        return jsonify({"msg": "Missing password parameter"}), 400

    if username != 'test' or password != 'test':
        return jsonify({"msg": "Bad username or password"}), 401

    # Identity can be any data that is json serializable
    ret = {'jwt': create_jwt(identity=username)}
```

```
    return jsonify(ret), 200

# Protect a view with jwt_required, which requires a valid jwt
# to be present in the headers.
@app.route('/protected', methods=['GET'])
@jwt_required
def protected():
    # Access the identity of the current user with get_jwt_identity
    return jsonify({'hello_from': get_jwt_identity()}), 200

if __name__ == '__main__':
    app.run()
```

To access a `jwt_required` protected view, all we have to do is send in the JWT with the request. By default, this is done with an authorization header that looks like:

```
Authorization: Bearer <access_token>
```

We can see this in action using CURL:

```
$ curl http://localhost:5000/protected
{
  "msg": "Missing Authorization Header"
}

$ curl -H "Content-Type: application/json" -X POST \
-d '{"username":"test","password":"test"}' http://localhost:5000/login
{
  "jwt": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJleHAiOiJlMDM1OTk3MTgsImhhdCI6MTUwMzU5NjExOCwibmJmIjoxNTAzNTk2MTE4LCJzdWIiOiJ0ZXN0In0.G2GnN9NgvvmSKgRDGok00jAyDWkG_qCn4FTxSfPUXDY"
}

$ export ACCESS="eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJleHAiOiJlMDM1OTk3MTgsImhhdCI6MTUwMzU5NjExOCwibmJmIjoxNTAzNTk2MTE4LCJzdWIiOiJ0ZXN0In0.G2GnN9NgvvmSKgRDGok00jAyDWkG_qCn4FTxSfPUXDY"

$ curl -H "Authorization: Bearer $ACCESS" http://localhost:5000/protected
{
  "hello_from": "test"
}
```

NOTE: Remember to change the `JWT_SECRET_KEY` on your application, and insure that no one is able to view it. The json web tokens are signed with the secret key, so if someone gets that, they can create arbitrary tokens, and in essence log in as any user.

CHAPTER 3

Changing JWT Claims

You may want to change the claims that are stored in the created JWTs. This can be done with the `@jwt.jwt_data_loader` decorator, and the jwt can be accessed in your protected endpoints with the `get_jwt()` function.

```
from datetime import datetime

from flask import Flask, jsonify, request, current_app
from flask_jwt_simple import (
    JWTManager, jwt_required, create_jwt, get_jwt
)

app = Flask(__name__)

app.config['JWT_SECRET_KEY'] = 'super-secret' # Change this!
jwt = JWTManager(app)

# Using the jwt_data_loader, we can change the values that
# will be present in the JWTs (that are made by the
# `create_jwt()` function). This will override everything
# currently in the token, so you will need to re-add
# the default claims (exp, iat, nbf, sub) if you still
# want them.
@jwt.jwt_data_loader
def add_claims_to_access_token(identity):
    if identity == 'admin':
        roles = 'admin'
    else:
        roles = 'peasant'

    now = datetime.utcnow()
    return {
        'exp': now + current_app.config['JWT_EXPIRES'],
        'iat': now,
        'nbf': now,
```

```
        'sub': identity,
        'roles': roles
    }

@app.route('/login', methods=['POST'])
def login():
    username = request.json.get('username', None)
    password = request.json.get('password', None)
    if username != 'test' or password != 'test':
        return jsonify({"msg": "Bad username or password"}), 401

    ret = {'jwt': create_jwt(username)}
    return jsonify(ret), 200

# In a protected view, you can get the full data encoded in the
# jwt with the `get_jwt()` function.
@app.route('/protected', methods=['GET'])
@jwt_required
def protected():
    jwt_data = get_jwt()
    if jwt_data['roles'] != 'admin':
        return jsonify(msg="Permission denied"), 403
    return jsonify(msg="Do not forget to drink your ovaltine")

if __name__ == '__main__':
    app.run()
```

Note: be careful of what you what data you put in the JWT. Any data in the JWT can be easily viewed with anyone who has access to the token. Make sure you don't put any sensitive information in them!

CHAPTER 4

Changing Default Behaviors

We provide what we think are sensible behaviors when attempting to access a protected endpoint. If the JWT is not valid for any reason (missing, expired, tampered with, etc) we will return json in the format of {'msg': 'why accessing endpoint failed'} along with an appropriate http status code (401 or 422). However, you may want to customize what you return in some situations. We can do that with the `jwt_manager` loader functions. An example of this looks like:

```
from flask import Flask, jsonify, request
from flask_jwt_simple import JWTManager, jwt_required, create_jwt

app = Flask(__name__)

app.config['JWT_SECRET_KEY'] = 'super-secret' # Change this!
jwt = JWTManager(app)

# Using the expired_token_loader decorator, we will now call
# this function whenever an expired but otherwise valid access
# token attempts to access an endpoint. There are other
# behaviors tht can be changed with these loader functions.
# Check the docs for a full list.
@jwt.expired_token_loader
def my_expired_token_callback():
    err_json = {
        "status": 401,
        "title": "Expired JWT",
        "detail": "The JWT has expired"
    }
    return jsonify(err_json), 401

@app.route('/login', methods=['POST'])
def login():
    username = request.json.get('username', None)
    password = request.json.get('password', None)
    if username != 'test' or password != 'test':
        return jsonify({"msg": "Bad username or password"}), 401
```

```
ret = {'access_token': create_jwt(username)}
return jsonify(ret), 200

@app.route('/protected', methods=['GET'])
@jwt_required
def protected():
    return jsonify({'hello': 'world'}), 200

if __name__ == '__main__':
    app.run()
```

Possible loader functions are:

Loader Dec- orator	Description	Function Arguments
ex- pired_token_loader	Function to call when an expired token ac- cesses a protected endpoint	None
in- valid_token_loader	Function to call when an invalid token ac- cesses a protected endpoint	Takes one argument - an error string indicating why the token is invalid
unauth- rized_loader	Function to call when a request with no JWT accesses a protected endpoint	Takes one argument - an error string indicating why the request is unauthorized

Configuration Options

You can change many options for how this extension works via

```
app.config['OPTION_NAME'] = new_option_value
```

JWT_HEADER_NAME	What header to look for the JWT in a request. Defaults to 'Authorization'
JWT_HEADER_TYPE	What type of header the JWT is in. Defaults to 'Bearer'. This can be an empty string, in which case the header contains only the JWT (instead of something like Authorization: Bearer <JWT>)
JWT_EXPIRES	How long a JWT created with <i>create_jwt()</i> should live before it expires. This takes a <i>datetime.timedelta</i> , and defaults to 1 hour
JWT_ALGORITHM	Which algorithm to sign the JWT with. See here for the options. Defaults to 'HS256'.
JWT_SECRET_KEY	The secret key needed for symmetric based signing algorithms, such as HS*.
JWT_PUBLIC_KEY	The public key needed for asymmetric based signing algorithms, such as RS* or ES*. PEM format expected.
JWT_PRIVATE_KEY	The private key needed for asymmetric based signing algorithms, such as RS* or ES*. PEM format expected.
JWT_IDENTITY_CLAIM	Which claim the <i>get_jwt_identity()</i> function will use to get the identity out of a JWT. Defaults to 'sub'.
JWT_DECODE_AUDIENCE	The audience you expect in a JWT when decoding it. Defaults to None. If this option differs from the 'aud' claim in a JWT, the <i>invalid_token_callback</i> is invoked.

In here you will find the API for everything exposed in this extension.

6.1 Configuring JWT Options

class flask_jwt_simple.**JWTManager** (*app=None*)

This object is used to hold the JWT settings and callback functions. Instances *JWTManager* are *not* bound to specific apps, so you can create one in the main body of your code and then bind it to your app in a factory function.

__init__ (*app=None*)

Create the JWTManager instance. You can either pass a flask application in directly here to register this extension with the flask app, or call `init_app` after creating this object

Parameters **app** – A flask application

init_app (*app*)

Register this extension with the flask app

Parameters **app** – A flask application

expired_token_loader (*callback*)

Sets the callback method to be called if an expired JWT is received

The default implementation will return json ‘{“msg”: “Token has expired”}’ with a 401 status code.

Callback must be a function that takes zero arguments.

invalid_token_loader (*callback*)

Sets the callback method to be called if an invalid JWT is received.

The default implementation will return json ‘{“msg”: <err>}’ with a 401 status code.

Callback must be a function that takes only one argument, which is the error message of why the token is invalid.

unauthorized_loader (*callback*)

Sets the callback method to be called if no JWT is received

The default implementation will return `{“msg”: “Missing Authorization Header”}` json with a 401 status code.

Callback must be a function that takes only one argument, which is the error message of why the token is invalid.

jwt_data_loader (*callback*)

Sets the callback method to be called for what data should be included in a JWT (with the `create_jwt()` function).

The default implementation will return the following data.

```
{
  'exp': now + current_app.config['JWT_EXPIRES'],
  'iat': now,
  'nbf': now,
  'sub': identity
}
```

Callback must be a function that takes only one argument, which is the identity of the user this JWT is for.

6.2 Protected endpoint decorators

`flask_jwt_simple.jwt_required(fn)`

If you decorate a view with this, it will ensure that the requester has a valid JWT before calling the actual view.

Parameters `fn` – The view function to decorate

`flask_jwt_simple.jwt_optional(fn)`

If you decorate a view with this, it will check the request for a valid JWT and put it into the Flask application context before calling the view. If no authorization header is present, the view will be called without the application context being changed. Other authentication errors are not affected. For example, if an expired JWT is passed in, it will still not be able to access an endpoint protected by this decorator.

Parameters `fn` – The view function to decorate

6.3 Utilities

`flask_jwt_simple.get_jwt()`

Returns the python dictionary which has all of the data in this JWT. If no JWT is currently present, an empty dict is returned

`flask_jwt_simple.get_jwt_identity()`

Returns the identity of the JWT in this context. If no JWT is present, None is returned.

`flask_jwt_simple.create_jwt(identity)`

Creates a new JWT.

Parameters `identity` – The identity of this token. This can be anything that is json serializable.

Returns A utf-8 encoded jwt.

`flask_jwt_simple.decode_jwt(encoded_token)`

Returns the decoded token from an encoded one. This does all the checks to insure that the decoded token is valid before returning it.

f

`flask_jwt_simple`, [13](#)

Symbols

`__init__()` (`flask_jwt_simple.JWTManager` method), 13

C

`create_jwt()` (in module `flask_jwt_simple`), 14

D

`decode_jwt()` (in module `flask_jwt_simple`), 14

E

`expired_token_loader()` (`flask_jwt_simple.JWTManager` method), 13

F

`flask_jwt_simple` (module), 13

G

`get_jwt()` (in module `flask_jwt_simple`), 14

`get_jwt_identity()` (in module `flask_jwt_simple`), 14

I

`init_app()` (`flask_jwt_simple.JWTManager` method), 13

`invalid_token_loader()` (`flask_jwt_simple.JWTManager` method), 13

J

`jwt_data_loader()` (`flask_jwt_simple.JWTManager` method), 14

`jwt_optional()` (in module `flask_jwt_simple`), 14

`jwt_required()` (in module `flask_jwt_simple`), 14

`JWTManager` (class in `flask_jwt_simple`), 13

U

`unauthorized_loader()` (`flask_jwt_simple.JWTManager` method), 13